# AI Engineer Data Collection & Intelligence Pipeline Architecture Document

This document describes the prototype architecture for collecting, normalizing, resolving and extracting AI ecosystem data.

## 1. High-Level Architecture

| Layer | Responsibility |
|---|---|
| Data Sources | Public APIs, RSS feeds and public datasets |
| Acquisition | Python HTTP clients, pagination and batching |
| Freshness | Publication timestamps and 24-hour filtering for jobs/news |
| Normalization | Standardized fields and schemas |
| Entity Resolution | Name normalization and canonical mapping |
| LLM Extraction | Structured JSON extraction with chunking and retries |
| Storage | CSV outputs with source URLs and timestamps |
| Version Control | Git and GitHub |

## 2. Data Acquisition

**Startups:** YC company data is collected into startup_data.csv; the current implementation produces 1,000 records.

**Products:** A public AI-tools dataset is collected into product_data.csv; missing fields are not fabricated.

**Research Papers:** arXiv metadata is collected in small batches to avoid large-response timeouts; the current implementation produces 1,000 records.

**Jobs:** A public jobs API is filtered by actual publication time to retain AI/ML jobs from the last 24 hours.

**News:** Google News RSS feeds are filtered by publication time; the current run produced 447 fresh AI-news articles.

## 3. Reliability and Error Handling

**413:** Large LLM inputs are split into smaller chunks. **429:** rate limits use retries with exponential backoff and deliberate request spacing. **Timeouts:** large research-paper requests use smaller batches, explicit timeouts and retries.

## 4. LLM Extraction

The LLM layer converts unstructured text into structured JSON-style records containing schema version, record type, source information, entity name and extracted data. Large inputs are chunked before processing.

## 5. Entity Resolution

Names are normalized by lowercasing, removing punctuation and common legal suffixes, and collapsing whitespace. Equivalent normalized names are grouped in entity_mapping_log.csv with source URLs and resolution methods.

## 6. Freshness

Jobs and news use UTC publication timestamps. Records older than current time minus 24 hours are excluded. Retained records store collection timestamps for auditing.

## 7. Storage and Traceability

The prototype stores normalized datasets as CSV files. Source information and collection timestamps support traceability and later migration to a database.

## 8. Scaling to 500k+ Records

For production scale, collectors should run as asynchronous workers behind a queue. Use pagination, batching, per-source concurrency limits, checkpoints, deduplication keys and idempotent writes. Raw data can be stored in object storage, normalized data in a database, and search workloads in an index. Scheduled workers can refresh time-sensitive sources frequently.

## 9. Production Flow

Source → Collector → Rate-limit/Retry Layer → Raw Data → Parser/Normalizer → Freshness Filter → Entity Resolution → LLM Extraction → Validation → Storage → Search/API → Monitoring.

## 10. Current Deliverables

| Deliverable | Status |
|---|---|
| startup_data.csv | 1,000 startup records |
| product_data.csv | Product dataset generated |
| research_papers.csv | 1,000 research-paper records |
| jobs.csv | Fresh AI/ML jobs |
| news.csv | 447 fresh AI-news articles |
| entity_mapping_log.csv | Entity-resolution log |
| Python collectors | Implemented |
| LLM extraction pipeline | Implemented with chunking/retry design |
| GitHub | Changes committed and pushed |